Modern Regression

Lasso, Ridge, and Elastic Net

Jyotishka Datta

2023-03-29

Source

The R codes for the baseball example and the data are taken from the ISLR book by James, Witten, Hastie and Tibshirani.

This is a low-dimensional example, which means $p < n$. We will see how LASSO and Ridge compares with OLS on this data-set.

We will see a high-dimensional example after this. Of course, there isn't a unique OLS for high-dimensional data.

Baseball data

Major League Baseball Data from the 1986 and 1987 seasons.

```
library(ISLR)
#fix(Hitters)
names(Hitters)
```

```
##  [1] "AtBat"     "Hits"      "HmRun"     "Runs"      "RBI"
##  [7] "Years"     "CAtBat"    "CHits"     "CHmRun"    "CRuns"
## [13] "CWalks"    "League"    "Division"  "PutOuts"   "Assists"
## [19] "Salary"    "NewLeague"
```

```
dim(Hitters)
```

```
## [1] 322  20
```

Variables

- `AtBat`: Number of times at bat in 1986, `Hits`: Number of hits in 1986
- `HmRun`: Number of home runs in 1986, `Runs`: Number of runs in 1986
- `RBI`: Number of runs batted in in 1986, `Walks`: Number of walks in 1986.
- `Years`: Number of years in the major leagues
- `CAtBat`: Number of times at bat during his career
- `CHits`: Number of hits during his career
- `CHmRun`: Number of home runs during his career
- `CRuns`: Number of runs during his career
- `CRBI`: Number of runs batted in during his career
- `CWalks`: Number of walks during his career
- `League`: A factor with levels A and N indicating player's league at the end of 1986
- `Division`: A factor with levels E and W indicating player's division at the end of 1986
- `PutOuts`: Number of put outs in 1986
- `Assists`: Number of assists in 1986
- `Errors`: Number of errors in 1986
- `Salary`: 1987 annual salary on opening day in thousands of dollars
- `NewLeague`: A factor with levels A and N indicating player's league at the beginning of 1987

Remove NA's

Salary is missing for some players. The `na.omit()` function removes all of the rows that have missing values in any variable.

```
sum(is.na(Hitters$Salary))
```

```
## [1] 59
```

```
Hitters=na.omit(Hitters)
dim(Hitters)
```

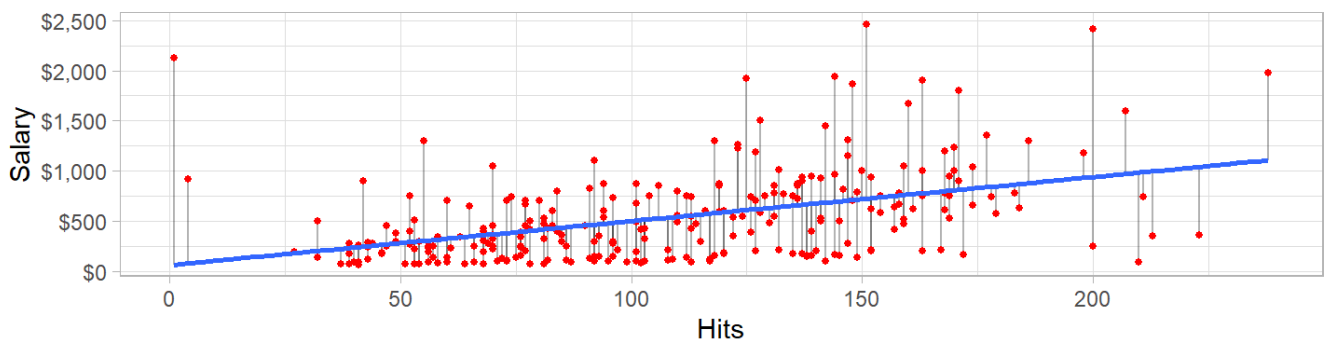
```
## [1] 263  20
```

Load a few extra packages

```r
## prevent scientific notation
options(scipen = 100, digits = 4)

# Set the graphical theme
ggplot2::theme_set(ggplot2::theme_light())

# Helper packages
library(recipes)  # for feature engineering

# Modeling packages
library(glmnet)   # for implementing regularized regression
library(caret)    # for automating the tuning process

# Model interpretability packages
library(vip)      # for variable importance
```

Linear regression

- In OLS regression, the goal is to identify the hyperplane that minimizes the sum of squared errors (SSE or RSS) between the predicted and observed response values.
- This involves minimizing the vertical distance between the predicted (blue line) and observed (red dots) response values, which is represented by the grey lines.



Bet on Sparsity

- Classic regression models face issues with large number of features.
- Large number of features makes the model less interpretable.
- When $p > n$, there are infinite solutions to the OLS problem.
- It is useful and practical to assume that a smaller subset of features have the strongest effects.
- This assumption is known as the “bet on sparsity” principle, introduced by Hastie, Tibshirani, and Wainwright in 2015.

Feature selection

- Two approaches to address large number of features: hard thresholding and soft thresholding.
- Hard thresholding includes traditional linear model selection methods like forward selection and backward elimination.
- Soft thresholding is a more modern approach that slowly pushes irrelevant features toward zero, resulting in more accurate and interpretable models.
- Soft thresholding can zero out entire coefficients and is more efficient and scalable than hard thresholding.

Penalty

- Regularized regression model has an objective function similar to OLS, but with a penalty term P that constrains the size of coefficients.

$$\text{minimize}(\boldsymbol{RSS} + \boldsymbol{P})$$

- Coefficients can only increase if there is a comparable decrease in the residual sum of squares (RSS) or sum of squared errors (SSE).

- This concept applies to all GLM models, including logistic and Poisson regression, and some survival models, even though different GLM models may have different loss functions.

- The penalty parameter constrains the size of coefficients such that they can only increase if there is a comparable decrease in the model's loss function.

- There are three common penalty parameters we can implement:

1. Ridge: $\boldsymbol{P} = \lambda \sum_{i=1}^{p} \beta_i^2$.
2. Lasso (or LASSO): $\boldsymbol{P} = \lambda \sum_{i=1}^{p} |\beta_i|$.
3. Elastic net (or ENET): $\boldsymbol{P} = \lambda_1 \sum_{i=1}^{p} \beta_i^2 + \lambda_2 \sum_{i=1}^{p} |\beta_i|$.

Ridge Regression

- Ridge, Lasso, Elastic Net are all based on R package `glmnet`.
- Elastic Net Penalty:

$$P_{\lambda,\alpha}(\beta) = \lambda \left\{ \frac{(1-\alpha)}{2} \|\beta\|_2^2 + \alpha \|\beta\|_1^2 \right\}$$

- $\alpha = 0$ is Ridge, $\alpha = 1$ is Lasso, anything in between ($0 < \alpha < 1$ is called Elastic Net.
- $\lambda = 0$ is ordinary least squares.
- 'Elastic Net' is a mix of Lasso and Ridge - it works better when the predictor variables are 'grouped'.
- All the three methods can be fit using the `glmnet` package.

R commands

```
x=model.matrix(Salary~.,Hitters)[,-1]
y=Hitters$Salary
dim(x)
```

```
## [1] 263  19
```

```
length(y)
```

```
## [1] 263
```

Fitting Ridge

- We'll specify $\alpha = 0$ and calculate the ridge solution for 100 different values of λ, chosen by us.

```
library(glmnet)
grid=10^seq(10,-2,length=100)
ridge.mod=glmnet(x,y,alpha=0,lambda=grid)
dim(coef(ridge.mod))
```

```
## [1]  20 100
```

Dependence on λ

- When λ = 811130830.7897, check that the magnitude of the β coefficients are small.

```
ridge.mod$lambda[10]
```

```
## [1] 811130831
```

```
coef(ridge.mod)[,10]
```

```
##     (Intercept)           AtBat            Hits           HmRun
## 535.92355688461   0.00000067109   0.00000243436   0.00000980914
##             RBI           Walks           Years          CAtBat
##   0.00000434851   0.00000511793   0.00002093010   0.00000005762
##          CHmRun           CRuns            CRBI          CWalks
##   0.00000159921   0.00000042543   0.00000043906   0.00000046452
##       DivisionW         PutOuts         Assists          Errors
##  -0.00009625147   0.00000026880   0.00000004390  -0.00000020471
```

- and the value of $\sum_{i=1}^{p} \beta_i^2$ is also quite small.

```
sqrt(sum(coef(ridge.mod)[-1,10]^2))
```

```
## [1] 0.00009962
```

Dependence on λ

- Now, we decrease λ to λ = 11497.57.
- check that the magnitude of the β coefficients and the value of $\sum_{i=1}^{p} \beta_i^2$ has increased a bit.
- We are regularizing **less** than before!

```
ridge.mod$lambda[50]
```

```
## [1] 11498
```

```
coef(ridge.mod)[,50]
```

```
## (Intercept)       AtBat        Hits       HmRun        Runs
##  407.356050    0.036957    0.138180    0.524630    0.230702    0.
##       Walks       Years      CAtBat       CHits      CHmRun
##    0.289619    1.107703    0.003132    0.011654    0.087546    0.
##        CRBI      CWalks     LeagueN   DivisionW     PutOuts     A
##    0.024138    0.025015    0.085028   -6.215441    0.016483    0.
##      Errors  NewLeagueN
##   -0.020503    0.301434
```

```
sqrt(sum(coef(ridge.mod)[-1,50]^2))
```

```
## [1] 6.361
```

Dependence on λ

- Now, we have a small λ: λ = 0.01 :

```
ridge.mod$lambda[100]
```

```
## [1] 0.01
```

```
coef(ridge.mod)[,100]
```

```
## (Intercept)       AtBat        Hits       HmRun        Runs
##   164.11322    -1.97386     7.37772     3.93660    -2.19874    -(
##       Walks       Years      CAtBat       CHits      CHmRun
##     6.20038    -3.71403    -0.17510     0.21133     0.05629     1
##        CRBI      CWalks     LeagueN   DivisionW     PutOuts     A
##     0.70966    -0.79582    63.40493  -117.08244     0.28203     (
##      Errors  NewLeagueN
##    -3.42400   -25.99082
```

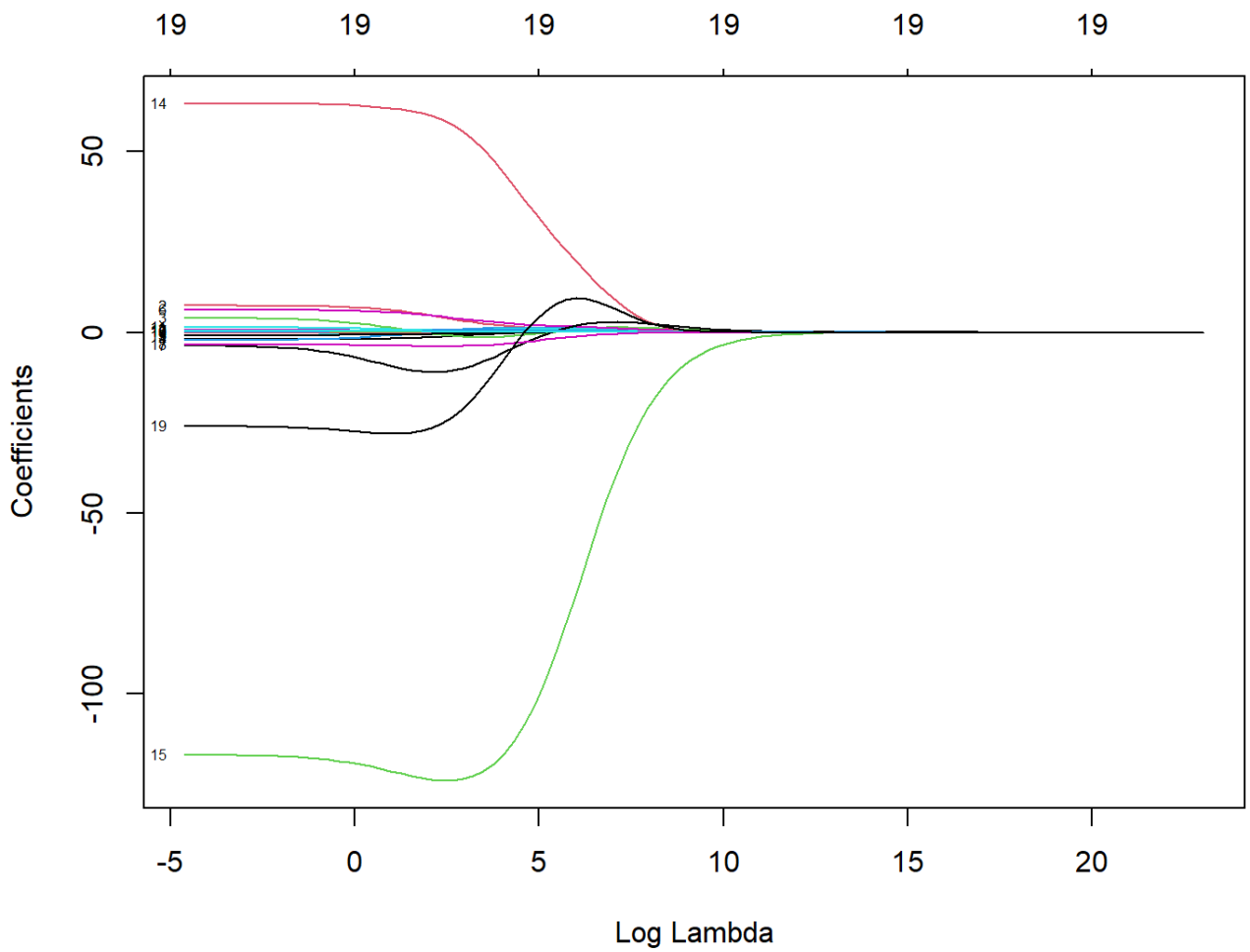
```
sqrt(sum(coef(ridge.mod)[-1,100]^2))
```

```
## [1] 136.2
```

- As you can see, the magnitudes of β_j's and the sum of square norm has increased signficantly.
- Somewhere in this spectrum lies the optimal β: find out using CV.

Solution path

```
plot(ridge.mod,xvar="lambda",label=TRUE)
```



Choosing the best solution

- We will use cross-validation to choose the best λ that minimizes the test error.
- We have discussed LOO-CV and k-fold CV in class. In general, LOO-CV has higher variance but k-fold is more stable.
- We'll first use a single validation set and then show CV.
- Also, note that if you are splitting randomly, you should **fix the random number generating seed** to reproduce your results.

Ridge with CV

- Now split the samples into a training set and a test set in order to estimate the test error of ridge regression.
- Two ways to split:
  1. Produce a random vector of `TRUE`, `FALSE` elements and select the observations corresponding to `TRUE` for the training data.
  2. Randomly choose a subset of numbers between 1 and n - these can then be used as the indices for the training observations.

Traing and Test

```
set.seed(1)
train=sample(1:nrow(x), nrow(x)/2)
test=(-train)
y.test=y[test]
dim(x[train,])
```

```
## [1] 131  19
```

```
dim(x[test,])
```

```
## [1] 132  19
```

Fit on training, evaluate on test

- Next we fit a ridge regression model on the training set, and evaluate its MSE on the test set, using $\lambda = 4$.
- The `predict()` function : here we get predictions for a test set, by replacing `type="coefficients"` with the `newx` argument.
- Specify `s = 4` to fix $\lambda = 4$.

```
ridge.mod=glmnet(x[train,],y[train],alpha=0,lambda=grid, thresh=1e-1
ridge.pred=predict(ridge.mod,s=4,newx=x[test,])
mean((ridge.pred-y.test)^2)
```

```
## [1] 142199
```

- Note: $\lambda = 4$ is an arbitrary value I have chosen for illustration. The optimal λ will be chosen by CV a few slides later.

Fit

The test MSE is 142199.1507

If we had instead simply fit a model with just an intercept, we would have predicted each test observation using the mean of the training observations:

```
mean((mean(y[train])-y.test)^2)
```

```
## [1] 224670
```

Another way

- We could also get the same result by fitting a ridge regression model with a very large value of λ. Note that `s=1e10` means $\lambda = 10^{10}$.

```
null.pred=predict(ridge.mod,s=1e10,newx=x[test,])
mean((null.pred-y.test)^2)
```

```
## [1] 224670
```

- So fitting a ridge regression model with $\lambda = 4$ leads to a much lower test MSE than fitting a model with just an intercept.

Obtaining least squres

- Is it better than just performing least squares regression?
- Ridge regression reduces to OLS for $\lambda = 0$.

```
ols.pred=predict(ridge.mod,s=0,newx=x[test,])
mean((ols.pred-y.test)^2)
```

```
## [1] 167790
```

- Ridge Regression MSE was 142199.1507.
- OLS has MSE 167789.7784

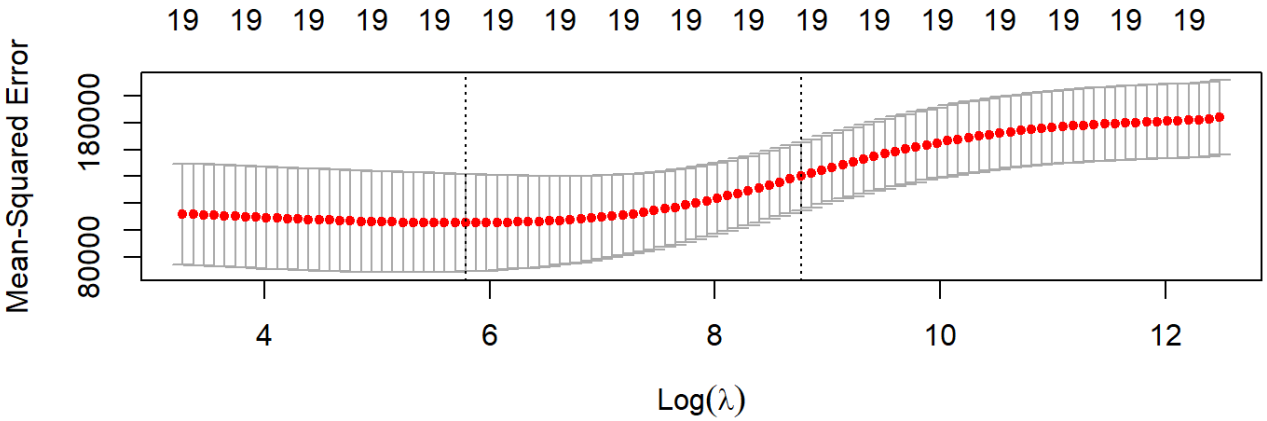
CV to choose λ

- In general, instead of arbitrarily choosing $\lambda = 4$, it would be better to use cross-validation to choose the tuning parameter.
- We can do this using the built-in cross-validation function, `cv.glmnet()`.
- By default, the function `cv.glmnet()` performs ten-fold cross-validation, though this can be changed using the argument `nfolds`.

Best lambda

Setting a random seed is important for reproducibility.

```
set.seed(1)
cv.out=cv.glmnet(x[train,],y[train],alpha=0)
plot(cv.out)
```



Best lambda

```
bestlam=cv.out$lambda.min
bestlam
```

```
## [1] 326.1
```

The value of λ that yields lowest CV error is 326.0828, very different from 4.

What happens to the MSE?

We should calculate the test error at this "best" λ and check if it's improving the fit.

```
bestlam=cv.out$lambda.min
bestlam
```

```
## [1] 326.1
```

```
ridge.pred=predict(ridge.mod,s=bestlam,newx=x[test,])
mean((ridge.pred-y.test)^2)
```

```
## [1] 139857
```

- It does reduce the test MSE.

The Coefficients

```
out=glmnet(x,y,alpha=0)
predict(out,type="coefficients",s=bestlam)[1:20,]
```

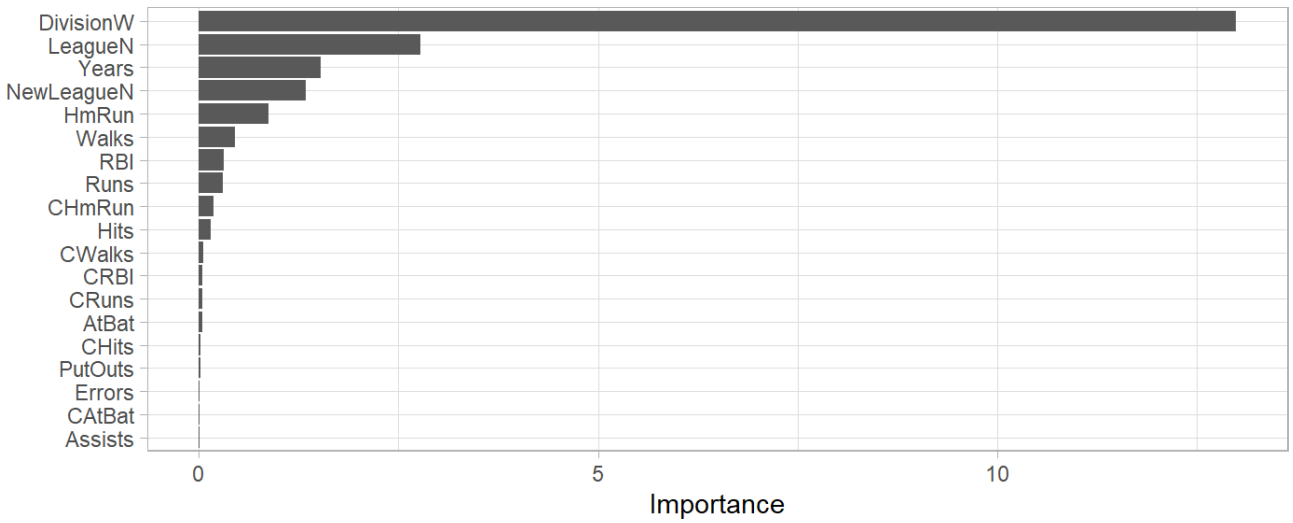
```
## (Intercept)       AtBat        Hits       HmRun        Runs
##    15.44383     0.07716     0.85912     0.60103     1.06369
##       Walks       Years      CAtBat       CHits      CHmRun
##     1.62445     1.35255     0.01135     0.05747     0.40680
##        CRBI      CWalks     LeagueN   DivisionW     PutOuts
##     0.12117     0.05299    22.09143   -79.04033     0.16620
##      Errors  NewLeagueN
##    -1.36093     9.12488
```

- As expected, none of the coefficients are zero : Ridge regression does not perform any variable selection.

Variable Importance

- Variable importance for regularized models provides a similar interpretation as in linear (or logistic) regression. Importance is determined by magnitude of the standardized coefficients

```
vip(cv.out, num_features = 20, bar = FALSE) + theme_light()
```



Lasso

Lasso Penalty

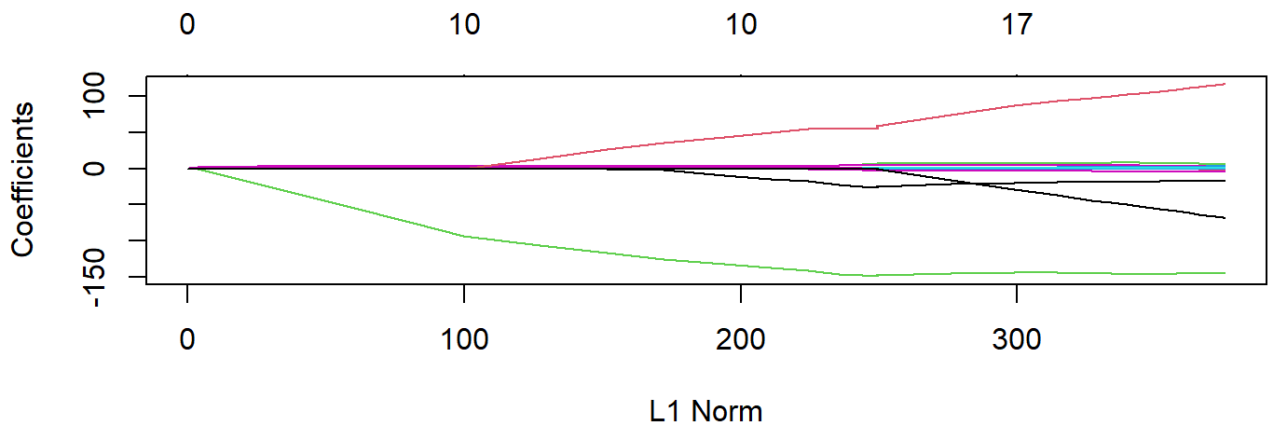
- Recall: Ridge, Lasso, Elastic Net are all based on R package `glmnet`.
- Elastic Net Penalty:

$$P_{\lambda,\alpha}(\beta) = \lambda \left\{ \frac{(1-\alpha)}{2} \|\beta\|_2^2 + \alpha \|\beta\|_1^2 \right\}$$

- $\alpha = 0$ is Ridge, $\alpha = 1$ is Lasso, anything in between is Elastic Net.
- All we need to do is change `alpha = 1` in `glmnet` function call.

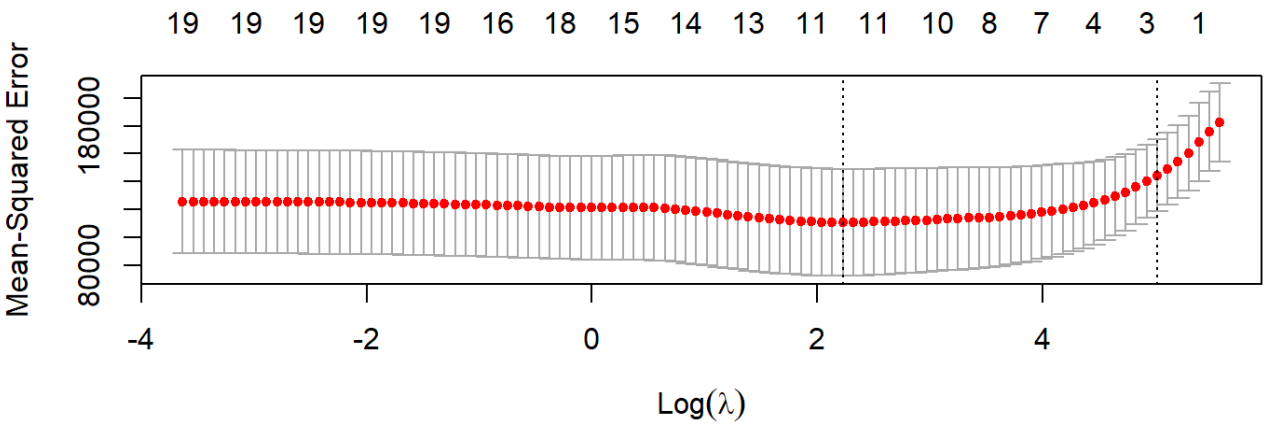
Lasso in R

```
lasso.mod =glmnet (x[train ,], y[train], alpha = 1, lambda = grid)
plot(lasso.mod)
```



Choosing lambda by 10-fold CV

```
set.seed (1)
cv.out =cv.glmnet( x[train ,], y[train], alpha = 1)
plot(cv.out)
```



Choosing lambda by 10-fold CV

```
bestlam =cv.out$lambda.min
lasso.pred=predict(lasso.mod, s = bestlam , newx = x[test,])
mean((lasso.pred - y.test)^2)
```

```
## [1] 143674
```

This is substantially lower than the test set MSE of the null model and of least squares, and very similar to the test MSE of ridge regression with λ chosen by cross-validation.

Sparse Model

```
out=glmnet (x,y, alpha = 1, lambda =grid)
lasso.coef=predict(out, type = "coefficients", s = bestlam )[1:20,]
lasso.coef
```

```
## (Intercept)       AtBat        Hits       HmRun        Runs
##     1.27479    -0.05497     2.18035     0.00000     0.00000
##       Walks       Years      CAtBat       CHits      CHmRun
##     2.29192    -0.33806     0.00000     0.00000     0.02825
##        CRBI      CWalks     LeagueN   DivisionW     PutOuts
##     0.41713     0.00000    20.28615  -116.16756     0.23752
##      Errors  NewLeagueN
##    -0.85629     0.00000
```

Sparse Model

Here we see that 12 of the 19 coefficient estimates are exactly zero. So the lasso model with λ chosen by cross-validation contains only seven variables.

```
lasso.coef[lasso.coef !=0]
```

```
## (Intercept)       AtBat        Hits       Walks       Years
##     1.27479    -0.05497     2.18035     2.29192    -0.33806     (
##       CRuns        CRBI     LeagueN   DivisionW     PutOuts
##     0.21628     0.41713    20.28615  -116.16756     0.23752    -(
```

Large p, small n

- Generate X, y with $n = 30, p = 60$ with a sparse β vector with 5 non-zero elements.
- Generate X matrix as $n \times p$ matrix with all entries from a $N(0, 1)$ distribution.
- Take $\beta = (3, \ldots, 3, 0, \ldots, 0)$, and a small error distribution $\epsilon \sim N(0, 0.1)$.

Generating the data

```
library(Matrix)
set.seed(12345)
n = 30;p = 60
p1 = 5
beta <- c(rep(3,p1),rep(0,p-p1))
x=scale(matrix(rnorm(n*p),n,p))
eps=rnorm(n,mean=0,sd = 0.1)
fx = x %*% beta
y=drop(fx+eps)
```

- Our goal is to see if Lasso can recover the 5 non-zero β's correctly.

Ordinary Least Squares

- The OLS Solution will not work here, as expected.

```
xtx = t(x)%*%x
y1 <- t(x) %*% y
betahat.1 <- solve(xtx,y1)
```

```
## Error in solve.default(xtx, y1): system is computationally singul
```

Does Pseudo-Inverse Work?

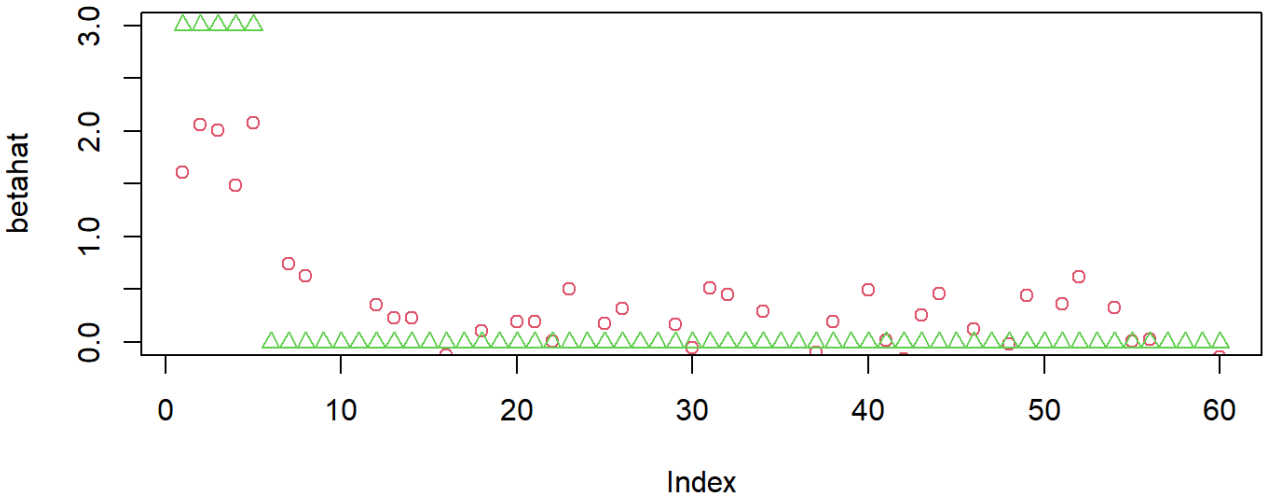
The Moore-Penrose g-inverse is unstable and does not recover the true sparsity pattern.

```
library(MASS)
xtx = t(x)%*%x
y1 <- t(x) %*% y
xtx <- as.matrix(xtx)
betahat <- ginv(xtx) %*% y1
t(betahat)
```

```
##       [,1]  [,2]  [,3]  [,4]  [,5]    [,6]   [,7]   [,8]    [,9]
## [1,] 1.609 2.055 2.008 1.483 2.075 -0.3615 0.7423 0.6313 -0.5369
##        [,11]  [,12]  [,13] [,14]   [,15]   [,16]   [,17]  [,18]
## [1,] -0.4607 0.3531 0.2274 0.226 -0.4165 -0.1229 -0.9068 0.1028 -
##      [,21]    [,22]  [,23]   [,24]  [,25]  [,26]   [,27]   [,28]
## [1,] 0.197 0.006585 0.5017 -0.1722 0.1779 0.3186 -0.3519 -0.6238
##         [,30]  [,31]  [,32]   [,33]  [,34]   [,35]   [,36]    [,3
## [1,] -0.05045 0.5132 0.4553 -0.2887 0.2955 -0.5663 -0.1818 -0.097
##        [,39]  [,40]   [,41]  [,42]  [,43]  [,44]  [,45]  [,46]
## [1,] -0.5981 0.4919 0.02024 -0.158 0.2579 0.4575 -1.059 0.1265 -0
##       [,49]   [,50]  [,51]  [,52]   [,53]  [,54]    [,55]   [,56]
## [1,] 0.4381 -0.7195 0.3673 0.6158 -0.5537 0.3277 0.006263 0.02666
##        [,58]   [,59]   [,60]
## [1,] -0.7236 -0.1992 -0.1381
```

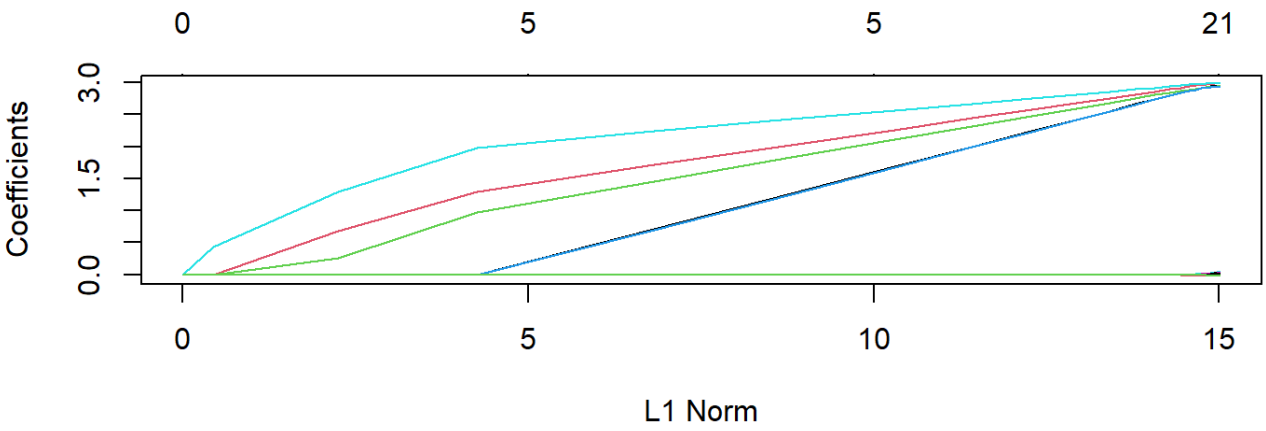
The g-Inverse estimate

```
plot(betahat,col=2,ylim=c(0,3), pch = 1)
points(beta,col= 3, pch = 2)
legend(75,2,c("G-inv","True"),col=c(2,3),pch=c(1,2))
```



Enter Lasso

```
grid=10^seq(10,-2,length=100)
lasso.mod =glmnet(x,y,alpha =1,lambda =grid)
plot(lasso.mod)
```

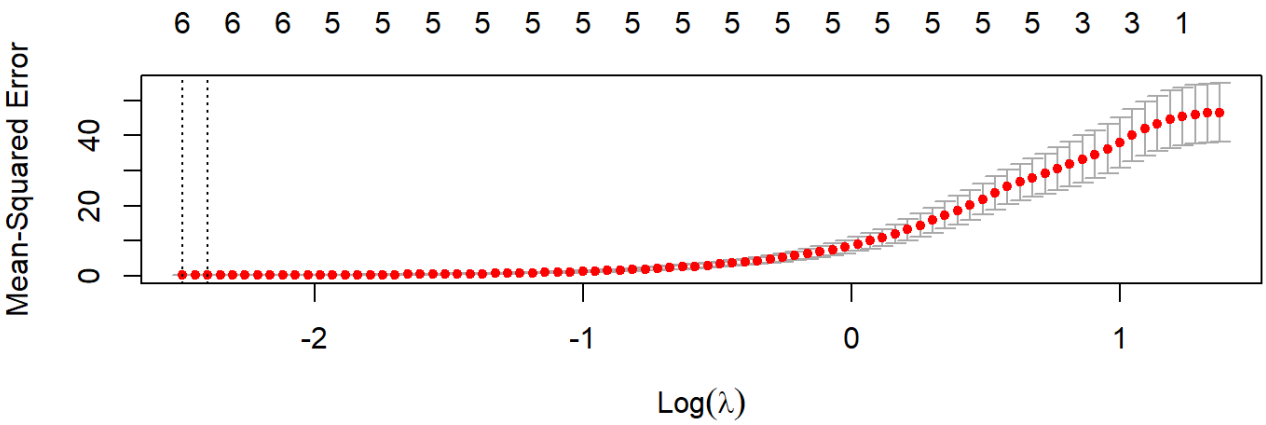


Choosing Lambda

```
cv.out =cv.glmnet(x,y,alpha =1)
(bestlam =cv.out$lambda.min)
```

```
## [1] 0.08296
```

```
plot(cv.out)
```



Sparse Solution

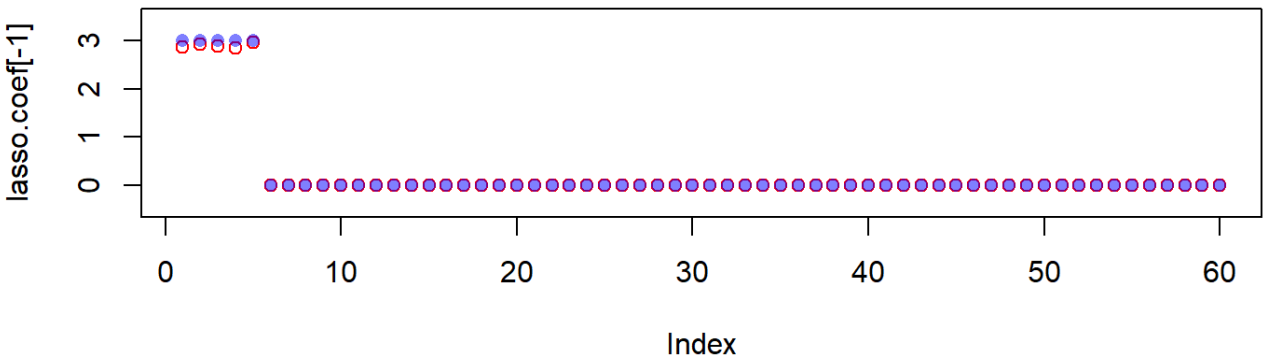
```
lasso.coef=predict(lasso.mod,type ="coefficients",s=bestlam)
lasso.coef[lasso.coef !=0]
```

```
## [1]  0.0146  2.8567  2.9295  2.8817  2.8510  2.9623 -0.0144
```

- Use `which` function to get rid of the warning message

```
lasso.coef[which(lasso.coef!=0)]
```

```
## [1]  0.0146  2.8567  2.9295  2.8817  2.8510  2.9623 -0.0144
```

Lasso is close to the truth

```
plot(lasso.coef[-1],col="red", pch = 1, ylim = c(-0.5,3.5))
points(beta,col= rgb(0,0,1,0.5),pch = 16)
legend(75,2,c("Lasso","True"),col=c("red","blue"),pch=c(17,16))
```